

CS3237 Lab 3 Statistical Methods

[Student Number|Name | |-----|-----| | Wira Azmoon Ahmad | | A0197788X | R Ramana |

In this lab you will also be do some experiments to familiarize yourself with the linear regression, Naive Bayes and Support Vector Machine library in SciKit Learn.

Please work together as a team of 2 to complete this lab. You will need to submit ONE copy of this notebook per team, but please fill in the names of both team members above. This lab is worth 40 marks (but it is still 5% CA like your other labs):

DO NOT SUBMIT MORE THAN ONE COPY OF THIS LAB!

Let's now begin using statistical techniques in SciKit Learn.

1. SciKit Learn Hands-on

We will now run some experiments to familiarize you with the statistical learning tools in SciKit Learn.

1.1 Linear Regression

Let's begin by playing around with the linear regression we did for the Boston Housing Dataset during the lecture.

1.1.1 Finding Better Correlations

Question 0: (4 marks)

In the lecture we took a quick look at correlating housing prices and poverty levels. Using the code cell below:

1. Recreate the regression example from the lecture Jupyter Notebook. Set `random_state` to 0 for `train_test_split`.
2. Add code to find the correlation between housing prices and the other independent variables in the dataset. Leave out CHAS (Charles River dummy variable) and MEDV.
3. As before save 33% of the data for testing and set `random_state` to 0.
4. Create a new simple (single independent variable) regression model with the independent variable with the absolute highest correlation. If poverty levels is the highest, then choose the next highest.
5. Compute and print the MSE for training data and testing data, and answer the questions after the code block.

```
In [ ]: """
        Enter your code for part 1.1.1 here, and answer the questions
        after this code cell.
        """
import numpy as np
import pandas as pd
import scipy.stats as stats
import sklearn

from sklearn import metrics
from sklearn.model_selection import train_test_split
```

```
In [ ]: from sklearn.datasets import load_boston
boston = load_boston()
```

```
c:\Users\wiraa\miniconda3\envs\cs3237\lib\site-packages\sklearn\utils\deprecation.py:87: FutureWarning: Function load_boston is deprecated; `load_boston` is deprecated in 1.0 and will be removed in 1.2.
```

The Boston housing prices dataset has an ethical problem. You can refer to the documentation of this function for further details.

The scikit-learn maintainers therefore strongly discourage the use of this dataset unless the purpose of the code is to study and educate about ethical issues in data science and machine learning.

In this special case, you can fetch the dataset from the original source::

```
import pandas as pd
import numpy as np

data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep="\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])
target = raw_df.values[1::2, 2]
```

Alternative datasets include the California housing dataset (i.e. :func:`~sklearn.datasets.fetch\_california\_housing`) and the Ames housing dataset. You can load the datasets as follows::

```
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
```

for the California housing dataset and::

```
from sklearn.datasets import fetch_openml
housing = fetch_openml(name="house_prices", as_frame=True)
```

for the Ames housing dataset.
warnings.warn(msg, category=FutureWarning)

```
In [ ]: bos = pd.DataFrame(boston.data)
bos.head()
```

```
Out[ ]:
```

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33

```
In [ ]: bos.columns = boston.feature_names
bos.head()
```

```
Out[ ]:
```

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	6.575	65.2	4.0900	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	6.421	78.9	4.9671	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	7.185	61.1	4.9671	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	6.998	45.8	6.0622	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	7.147	54.2	6.0622	3.0	222.0	18.7	396.90	5.33

```
In [ ]: crime_indus = bos['CRIM'].corr(bos['INDUS'])
pov_age = bos['LSTAT'].corr(bos['AGE'])
crime_tax = bos['CRIM'].corr(bos['TAX'])

print("corr crime/industry: %3.3f, corr poverty/age: %3.3f, corr crime/tax: %3.3f"
      % (crime_indus, pov_age, crime_tax))
```

corr crime/industry: 0.407, corr poverty/age: 0.602, corr crime/tax: 0.583

```
In [ ]: bos['PRICE'] = boston.target
print("Correlation between property prices and poverty levels: %3.3f"
      % bos['PRICE'].corr(bos['LSTAT']))
```

Correlation between property prices and poverty levels: -0.738

```
In [ ]: for feat in boston.feature_names:
        if feat == 'CHAS': continue
        print(f"Correlation between property prices and {feat}: {bos['PRICE'].corr(bos[feat])}")
```

Correlation between property prices and CRIM: -0.3883046085868113
Correlation between property prices and ZN: 0.3604453424505428
Correlation between property prices and INDUS: -0.48372516002837274
Correlation between property prices and NOX: -0.42732077237328264
Correlation between property prices and RM: 0.6953599470715396
Correlation between property prices and AGE: -0.3769545650045963
Correlation between property prices and DIS: 0.24992873408590394
Correlation between property prices and RAD: -0.3816262306397777
Correlation between property prices and TAX: -0.468535933567767
Correlation between property prices and PTRATIO: -0.5077866855375618
Correlation between property prices and B: 0.3334608196570665
Correlation between property prices and LSTAT: -0.7376627261740147

```
In [ ]: from sklearn.linear_model import LinearRegression

Y = bos['PRICE'].values.reshape(-1, 1)
Y_train, Y_test = train_test_split(Y, test_size=0.33, random_state=0)
```

RM: Average number of rooms per dwelling is the strongest correlated after poverty levels.

```
In [ ]: def find_mse_single_var(variable: str):
        lm = LinearRegression()
        # Reshape into column
        X = bos[variable].values.reshape(-1, 1)

        # Split data
        X_train, X_test = train_test_split(X, test_size=0.33, random_state=0)

        # Fit Linear Regression model
        lm.fit(X_train, Y_train)

        # Predict and evaluate
        Y_pred_train = lm.predict(X_train)
        Y_pred_test = lm.predict(X_test)
        train_mse = np.sqrt(metrics.mean_squared_error(Y_train, Y_pred_train))
        test_mse = np.sqrt(metrics.mean_squared_error(Y_test, Y_pred_test))

        print(f"{train_mse = :3.4f}, {test_mse = :3.4f}")

        return lm, train_mse, test_mse
```

```
In [ ]: find_mse_single_var('RM')
```

train_mse = 6.5675, test_mse = 6.6789

```
Out[ ]: (LinearRegression(), 6.567482862121251, 6.6788868044962175)
```

Answer the following questions between the ** markdowns so that your answers appear in bold.

Question 1: Which independent variable has the highest correlation? Did it have any effect on your training and test accuracy scores? Why or why not? (2 marks)

Answer: After poverty levels, average number of rooms per dwelling (RM) had the highest correlation. The training and test errors were slightly larger, since the best fit line had larger deviation from the data.

1.1.2 Creating Multivariate Linear Regressions

SciKit learn can create linear regression models with multiple independent variables, and in this section we are going to explore how to do this, and whether or not it makes a difference in our Boston Dataset.

One way to create a multivariate model is to:

1. Rank the independent variables by correlation, then create a linear model using the independent variable with the highest correlation. Measure the training and testing accuracy.
2. Add in the independent variable with the next highest correlation and create a new linear model. Measure the training and testing accuracy.
3. Stop when either accuracy score levels off or goes down.

Answer the following questions to help you along with creating your multivariate model:

Question 2: Explain what the following code fragment does. You may refer to NumPy and SciKit Learn documentation (2 marks)

```
bos['PRICE'].values.reshape(-1, 1)
```

Answer: It converts the price values to a column vector, -1 corresponding to any number of rows, automatically determined by numpy, and 1 corresponding to the number of columns. The number of rows will match how many values the array had before conversion.

Question 3: Consult the NumPy documentation: What does the 'concatenate' function do? In particular what does 'axis=1' do? (2 marks)

Answer: It joins a sequence of arrays along an existing axis, where axis=1 corresponds to joining the columns.

Question 4: Given your answers to Questions 2 and 3, what does the following code do? (2 marks)

```
import numpy as np

... Other code here ...

X1 = bos['INDUS'].values.reshape(-1, 1)
X2 = bos['CRIM'].values.reshape(-1, 1)
X = np.concatenate((X1, X2), axis = 1)
```

Answer: X1 and X2 are column vectors concatenated into matrix X. Rows of X corresponds to a data point, and columns of X corresponds to each feature.

Use the following code cell to follow the steps above to create models with one, two and three independent variables, printing the training and testing accuracy each time. Note that you have to run `train_test_split` for each model. Set the `random_state` parameter in `train_test_split` to 0 each time. (4 marks)

```
In [ ]: """
        Enter your code for part 1.1.2 here.
        """
def find_mse_multi_features(features: tuple[str, ...]):
    lm = LinearRegression()

    # Number of data points
    n = bos.shape[0]
    d = len(features)
    X = np.empty((n, d))

    # If following above steps, see feat_vecs
    # feat_vecs = []
    for i in range(d):
        feat = features[i]
        X[:, i] = bos[feat].values.reshape(-1)
        # feat_vecs.append(bos[feat].values.reshape(-1, 1))
    # X = np.concatenate(feat_vecs, axis = 1)
```

```

X_train, X_test = train_test_split(X, test_size=0.33, random_state=0)

# Fit Linear Regression model
lm.fit(X_train, Y_train)

# Predict and evaluate
Y_pred_train = lm.predict(X_train)
Y_pred_test = lm.predict(X_test)
train_mse = np.sqrt(metrics.mean_squared_error(Y_train, Y_pred_train))
test_mse = np.sqrt(metrics.mean_squared_error(Y_test, Y_pred_test))

print(f"Using feat(s) {features}")
print(f"RMSE for training data: {train_mse:3.4f}. RMSE for testing data: {test_mse:3.4f}.")
return lm, train_mse, test_mse

```

```

In [ ]: one_feat = ('PTRATIO',)
two_feat = ('LSTAT', 'RM')
three_feat = one_feat + two_feat
all_feats = (one_feat, two_feat, three_feat)

```

```

In [ ]: for feats in all_feats:
        find_mse_multi_features(feats)

```

```

Using feat(s) ('PTRATIO',)
RMSE for training data: 7.6882. RMSE for testing data: 8.3955.
Using feat(s) ('LSTAT', 'RM')
RMSE for training data: 5.4770. RMSE for testing data: 5.6286.
Using feat(s) ('PTRATIO', 'LSTAT', 'RM')
RMSE for training data: 4.9782. RMSE for testing data: 5.6952.

```

1.2 Creating a Naive Bayes Classifier

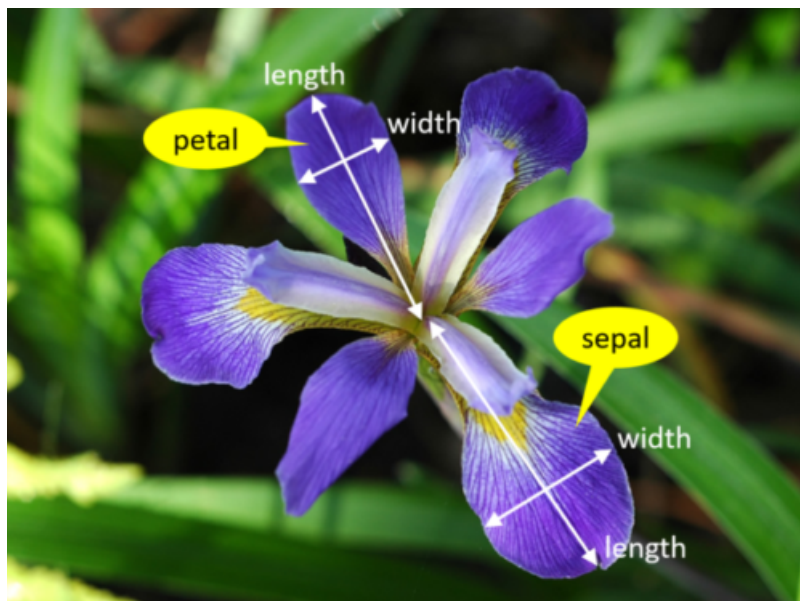
We will now look at how to create a Naive Bayes Classifier, and later on a Support Vector Machine classifier. We will also explore the use of *GridSearchCV* to optimize the choice of parameters for the SVC.

1.2.1 The Irises Dataset

In this lab we will use the irises dataset to classify four categories of irises (a species of flowers). We will consider four factors:

1. Sepal length in cm
2. Sepal width in cm
3. Petal length in cm
4. Petal width in cm

The image below shows what these mean:



The code cell below loads up the Iris dataset, prints it out, then scales it.

```
In [ ]: from sklearn.datasets import load_iris
        from sklearn.preprocessing import StandardScaler
        from sklearn.model_selection import train_test_split
        from sklearn.naive_bayes import GaussianNB
        import numpy as np

        iris_data = load_iris()
        print("Iris Data:")
        print(iris_data.data)
        scaler = StandardScaler()
        scaler.fit(iris_data.data)
        X = scaler.transform(iris_data.data)
        Y = iris_data.target
```

Iris Data:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 [5.8 4.  1.2 0.2]
 [5.7 4.4 1.5 0.4]
 [5.4 3.9 1.3 0.4]
 [5.1 3.5 1.4 0.3]
 [5.7 3.8 1.7 0.3]
 [5.1 3.8 1.5 0.3]
 [5.4 3.4 1.7 0.2]
 [5.1 3.7 1.5 0.4]
 [4.6 3.6 1.  0.2]
 [5.1 3.3 1.7 0.5]
 [4.8 3.4 1.9 0.2]
 [5.  3.  1.6 0.2]
 [5.  3.4 1.6 0.4]
 [5.2 3.5 1.5 0.2]
 [5.2 3.4 1.4 0.2]
 [4.7 3.2 1.6 0.2]
 [4.8 3.1 1.6 0.2]
 [5.4 3.4 1.5 0.4]
 [5.2 4.1 1.5 0.1]
 [5.5 4.2 1.4 0.2]
 [4.9 3.1 1.5 0.2]
 [5.  3.2 1.2 0.2]
 [5.5 3.5 1.3 0.2]
 [4.9 3.6 1.4 0.1]
 [4.4 3.  1.3 0.2]
 [5.1 3.4 1.5 0.2]
 [5.  3.5 1.3 0.3]
 [4.5 2.3 1.3 0.3]
 [4.4 3.2 1.3 0.2]
 [5.  3.5 1.6 0.6]
 [5.1 3.8 1.9 0.4]
 [4.8 3.  1.4 0.3]
 [5.1 3.8 1.6 0.2]
 [4.6 3.2 1.4 0.2]
 [5.3 3.7 1.5 0.2]
 [5.  3.3 1.4 0.2]
 [7.  3.2 4.7 1.4]
 [6.4 3.2 4.5 1.5]
 [6.9 3.1 4.9 1.5]
 [5.5 2.3 4.  1.3]
 [6.5 2.8 4.6 1.5]
 [5.7 2.8 4.5 1.3]
 [6.3 3.3 4.7 1.6]
 [4.9 2.4 3.3 1. ]
 [6.6 2.9 4.6 1.3]
 [5.2 2.7 3.9 1.4]
 [5.  2.  3.5 1. ]
 [5.9 3.  4.2 1.5]
 [6.  2.2 4.  1. ]
 [6.1 2.9 4.7 1.4]
 [5.6 2.9 3.6 1.3]
 [6.7 3.1 4.4 1.4]
 [5.6 3.  4.5 1.5]
 [5.8 2.7 4.1 1. ]
 [6.2 2.2 4.5 1.5]
 [5.6 2.5 3.9 1.1]
 [5.9 3.2 4.8 1.8]]
```

[6.1 2.8 4. 1.3]
[6.3 2.5 4.9 1.5]
[6.1 2.8 4.7 1.2]
[6.4 2.9 4.3 1.3]
[6.6 3. 4.4 1.4]
[6.8 2.8 4.8 1.4]
[6.7 3. 5. 1.7]
[6. 2.9 4.5 1.5]
[5.7 2.6 3.5 1.]
[5.5 2.4 3.8 1.1]
[5.5 2.4 3.7 1.]
[5.8 2.7 3.9 1.2]
[6. 2.7 5.1 1.6]
[5.4 3. 4.5 1.5]
[6. 3.4 4.5 1.6]
[6.7 3.1 4.7 1.5]
[6.3 2.3 4.4 1.3]
[5.6 3. 4.1 1.3]
[5.5 2.5 4. 1.3]
[5.5 2.6 4.4 1.2]
[6.1 3. 4.6 1.4]
[5.8 2.6 4. 1.2]
[5. 2.3 3.3 1.]
[5.6 2.7 4.2 1.3]
[5.7 3. 4.2 1.2]
[5.7 2.9 4.2 1.3]
[6.2 2.9 4.3 1.3]
[5.1 2.5 3. 1.1]
[5.7 2.8 4.1 1.3]
[6.3 3.3 6. 2.5]
[5.8 2.7 5.1 1.9]
[7.1 3. 5.9 2.1]
[6.3 2.9 5.6 1.8]
[6.5 3. 5.8 2.2]
[7.6 3. 6.6 2.1]
[4.9 2.5 4.5 1.7]
[7.3 2.9 6.3 1.8]
[6.7 2.5 5.8 1.8]
[7.2 3.6 6.1 2.5]
[6.5 3.2 5.1 2.]
[6.4 2.7 5.3 1.9]
[6.8 3. 5.5 2.1]
[5.7 2.5 5. 2.]
[5.8 2.8 5.1 2.4]
[6.4 3.2 5.3 2.3]
[6.5 3. 5.5 1.8]
[7.7 3.8 6.7 2.2]
[7.7 2.6 6.9 2.3]
[6. 2.2 5. 1.5]
[6.9 3.2 5.7 2.3]
[5.6 2.8 4.9 2.]
[7.7 2.8 6.7 2.]
[6.3 2.7 4.9 1.8]
[6.7 3.3 5.7 2.1]
[7.2 3.2 6. 1.8]
[6.2 2.8 4.8 1.8]
[6.1 3. 4.9 1.8]
[6.4 2.8 5.6 2.1]
[7.2 3. 5.8 1.6]
[7.4 2.8 6.1 1.9]
[7.9 3.8 6.4 2.]
[6.4 2.8 5.6 2.2]
[6.3 2.8 5.1 1.5]
[6.1 2.6 5.6 1.4]
[7.7 3. 6.1 2.3]
[6.3 3.4 5.6 2.4]
[6.4 3.1 5.5 1.8]
[6. 3. 4.8 1.8]
[6.9 3.1 5.4 2.1]
[6.7 3.1 5.6 2.4]
[6.9 3.1 5.1 2.3]
[5.8 2.7 5.1 1.9]


```
[6.8 3.2 5.9 2.3]
[6.7 3.3 5.7 2.5]
[6.7 3. 5.2 2.3]
[6.3 2.5 5. 1.9]
[6.5 3. 5.2 2. ]
[6.2 3.4 5.4 2.3]
[5.9 3. 5.1 1.8]]
```

Answer the following questions:

Question 5: What does 'StandardScaler' do? What other types of scalers are available? What is the advantage of scaling your inputs? (2 marks)

Answer: StandardScaler standardizes features by removing the mean and scaling the data to unit variance. There is **MinMaxScaler** (rescales data so they fit the range 0 to 1); **MaxAbsScaler** (similar to MinMaxScaler but uses different ranges - if only negative values, the range -1 to 0 is used, if data has only positive values it uses 0 to 1, and if it contains both negative and positive values it uses -1 to 1); **RobustScaler** (scales based on percentiles), **PowerTransformer** (adds a power transformation to each feature to make the data more Guassian like to stabilize variance and minimize skewed data), **QuantileTransformer** (non-linear transformation so the probability density function of each feature will be mapped to a Guassian distribution), **Normalizer** (scales the vector to have a unit norm, independent of the sample distribution). If one variable varies more than the other (e.g. weight vs height of males) due to the differing units, the algorithm may be biased towards a feature set if they are not scaled. By scaling we are able to account for the difference in units and consequently the differing variances of the feature sets.

1.2.2 Creating a Naive Bayes Classifier Model

Recall that there are three major types of Naive Bayes classifiers:

1. Gaussian
2. Multinomial
3. Bernoulli

Question 6: What type of model should we use here? Why? (2 marks)

Answer: Gaussian. We are working with length and width which are in cm and varies and are continuous. Multinomial works with frequencies of occurrence and Bernoulli works with IF an event occurred (boolean occurrences). As the petal and sepal length varies, they are likely to follow a normal distribution. Hence we choose a Gaussian Naive Bayes Classifier.

Now complete the code in the code cell below, following these specifications: (4 marks)

1. Set aside 20% of the data for testing.
2. Use the appropriate type of Naive Bayes Classifier, adding in whatever import statements you require here.
3. Print out the training and testing accuracies.

```
In [ ]: """
        Enter your code for part 1.2.2 here.
        """
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
import numpy as np

iris_data = load_iris()
print("Iris Data:")
# print(iris_data.data)
sc = StandardScaler()

X = iris_data.data
Y = iris_data.target

# Random state to ensure reproducibility
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0)

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

gaussian = GaussianNB()

gaussian.fit(X_train, Y_train)
Y_pred_train = gaussian.predict(X_train)
Y_pred_test = gaussian.predict(X_test)
# print(Y_pred)

from sklearn.metrics import accuracy_score
print ("Training Accuracy : ", accuracy_score(Y_train, Y_pred_train))
print ("Test Accuracy : ", accuracy_score(Y_test, Y_pred_test))
```

```
Iris Data:
Training Accuracy : 0.95
Test Accuracy : 0.9666666666666667
```

1.2.3 Using Pipelines

In the Naive Bayes Jupyter Notebook included with your Lecture 4 slides, we used a *Pipeline* object to simplify our code. Using that example as a guide, rewrite your code above to use *Pipeline*. Some things to note:

1. The code will not be exactly the same (it will be much simpler). For example we are not using a *CountVectorizer* nor a *TfidfTransformer*. So just follow the principle. Remember to put your *StandardScaler* into the *Pipeline*.
2. When doing 'fit' on your model, you should input the *_original_* data, not the scaled one, since we are incorporating the *StandardScaler* as part of our *Pipeline*.

Hint: Section 1.3.2 below shows you how to create a Pipeline for SVM

Use the code cell below to enter your new version using Pipelines. Remember to print out your training and testing accuracies. (4 marks)

```
In [ ]: """
        Enter your code for part 1.2.3 here.
        """

from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.pipeline import Pipeline
import numpy as np

iris_data = load_iris()
print("Iris Data:")
sc = StandardScaler()

X = iris_data.data
Y = iris_data.target

# Random state to ensure reproducibility
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0)

gNB = Pipeline([('scaler', StandardScaler()),
                ('clf', GaussianNB())])

gNB.fit(X_train, Y_train)

Y_pred_train = gNB.predict(X_train)
Y_pred_test = gNB.predict(X_test)

from sklearn.metrics import accuracy_score
print ("Training Accuracy : ", accuracy_score(Y_train, Y_pred_train))
print ("Test Accuracy : ", accuracy_score(Y_test, Y_pred_test))
```

Iris Data:
Training Accuracy : 0.95
Test Accuracy : 0.9666666666666667

1.3 Creating a Support Vector Machine Classifier

We will now create an SVM to perform our classification. There are two major SVM classifiers provided with SciKit Learn:

1. LinearSVC: An SVM that uses a linear decision boundary to classify.
2. SVC: An SVM that offers a wider variety of classification boundaries: Radial Basis Function (so-called 'kernel'), sigmoid, polynomials, and of course a linear boundary.

1.3.1 Creating a Linear SVM

Using your code from 1.2.3 as a guide, create a new Pipeline to train a LinearSVC with the following parameters:

- max_iter: 100000
- loss: hinge
- penalty: l2 (Note: This is 'el-two', and not 'twelve')

Use the code cell below to implement your SVM, printing out your training and testing accuracies. Please consult the SciKit Learn documentation on what these parameters mean. (4 marks)

```
In [ ]: """
        Enter your code for part 1.3.1 here.
        """
from sklearn.svm import LinearSVC
from sklearn.pipeline import Pipeline

def make_classifier_print_error(loss: str = 'hinge', penalty: str = 'l2'):
    X = iris_data.data
    Y = iris_data.target

    # Random state to ensure reproducibility
    X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0)

    clf = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', LinearSVC(random_state=0, max_iter=100000, loss=loss, penalty=penalty))
    ])
    clf.fit(X_train, Y_train)

    # Predict and evaluate
    Y_train_pred = clf.predict(X_train)
    Y_test_pred = clf.predict(X_test)
    train_acc = np.mean(Y_train_pred == Y_train)
    test_acc = np.mean(Y_test_pred == Y_test)

    print(f"Using {loss = }, {penalty = }")

    print(f"SVM Train Accuracy: {train_acc:3.2f}")
    print(f"SVM Test Accuracy: {test_acc:3.2f}\n")
    return clf, train_acc, test_acc
```

```
In [ ]: loss_types = ('hinge', 'squared_hinge')
penalty_types = ('l1', 'l2')

for s in loss_types:
    for p in penalty_types:
        try:
            make_classifier_print_error(s, p)
        except ValueError:
            print(f'Cannot use {s} loss with {p} penalty\n')
```

Cannot use hinge loss with l1 penalty

```
Using loss = 'hinge', penalty = 'l2'  
SVM Train Accuracy: 0.91  
SVM Test Accuracy: 0.87
```

Cannot use squared_hinge loss with l1 penalty

```
Using loss = 'squared_hinge', penalty = 'l2'  
SVM Train Accuracy: 0.94  
SVM Test Accuracy: 0.97
```

Question 7: Play around with the loss and penalty parameters. E.g. try an 'l1' penalty with hinge loss, or 'l1' penalty with squared hinge loss. Does 'l2' work with the squared hinge loss function? Record your training and testing accuracies below (2 marks)

Answer: 'l1' does not work with 'hinge', and 'l2' does not work with 'squared_hinge'. Using loss = 'hinge', penalty = 'l2', Training Acc: 0.91, Test Acc: 0.87. Using loss = 'squared_hinge', penalty = 'l2', Training Acc: 0.94. Testing Acc: 0.97.

1.3.2 Autotuning Hyperparameters

In Question 7 you have played around with some of the hyperparameters for LinearSVC and may have found that it gives you different accuracy results. Selecting the right hyperparameters is always a challenge, but thankfully SciKit Learn gives us a very useful tool called "GridSearchCV". In the example below we see how to tweak the 'C' parameter, which controls penalties applied to the SVM parametrs, to various values of between 1 and 10. GridSearchCV will then select the C value that gives us the best possible training accuracy:

```
from sklearn import svm  
from sklearn.pipeline import Pipeline  
from sklearn.model_selection import GridSearchCV  
  
params = {'C':[1,10]}  
  
svm_pipe_2 = Pipeline([('scaler', StandardScaler()),  
                        ('svm', GridSearchCV(svm.LinearSVC(max_iter = 100000), params)),  
                        ], )  
svm_pipe_2.fit(X_train_1, Y_train_1)  
  
Y_train_pred_1 = svm_pipe_2.predict(X_train_1)  
Y_test_pred_1 = svm_pipe_2.predict(X_test_1)  
  
print("SVM Train Accuracy: %3.2f" % np.mean(Y_train_pred_1 == Y_train_1))  
print("SVM Test Accuracy: %3.2f" % np.mean(Y_test_pred_1 == Y_test_1))
```

Note that the code above will not run because it's missing several variables, including X_train_1, etc. Notice that GridSearchCV is created in the Pipeline and takes svm.LinearSVC as a parameter.

The "param" variable is a dictionary that specifies which parameters to tune (in this case just simply 'C'), and what values to use (here [1, 10] means to use between 1 and 10). You can also specify labels instead of numeric values. E.g.:

```
params = {'kernel':('linear', 'poly')}
```

GridSearchCV will try 'linear' and 'poly', specified in the tuple after 'kernel', when tuning the SVM.

Use the code cell below to create a Pipeline that uses SVC (instead of LinearSVC), and applies GridSearchCV to tune the following hyperparameters:

- C: From 1 to 10 as before
 - kernel: 'linear', 'poly', 'rbf', 'sigmoid'
 - decision_function_shape: 'ovr', 'ovo'
-

Question 8: Consult the SVC documentation and write down below what each hyperparameter means. Also what is a 'decision function shape', and what is the difference between 'ovr' and 'ovo' in our decision function shape? (2 marks)

Answer: C is the regularization parameter, where the strength of regularization is inversely proportional to C. The kernel function is used to transform the data with the goal of making it linearly separable, typically in higher dimensions. The decision_function_shape is whether to use one-vs-rest (ovr) or one-vs-one (ovo) classifier, with shapes (n_samples, n_classes) and (n_samples, n_classes * (n_classes - 1) / 2) respectively.

Remember to print out the training and testing accuracies. (4 marks)

```
In [ ]: """
Enter your code for part 1.3.2 here.
"""

from sklearn import svm
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

def make_gridCV_classifier_print_errors():
    X = iris_data.data
    Y = iris_data.target

    X_train_1, X_test_1, Y_train_1, Y_test_1 = train_test_split(X, Y, test_size=0.2, random_state=0)

    # Hyperparameters to tune
    params = {'C':[1,10], 'kernel': ('linear', 'poly', 'rbf', 'sigmoid'), 'decision_function_shape': ('ovr', 'ov')}

    svm_pipe_2 = Pipeline([('scaler', StandardScaler()),
                           ('svm', GridSearchCV(svm.SVC(max_iter = 100000), params)), ])
    svm_pipe_2.fit(X_train_1, Y_train_1)

    Y_train_pred_1 = svm_pipe_2.predict(X_train_1)
    Y_test_pred_1 = svm_pipe_2.predict(X_test_1)

    train_acc = np.mean(Y_train_pred_1 == Y_train_1)
    test_acc = np.mean(Y_test_pred_1 == Y_test_1)

    print(f"SVM Train Accuracy: {train_acc:3.2f}")
    print(f"SVM Test Accuracy: {test_acc:3.2f}")
    return svm_pipe_2, train_acc, test_acc
```

```
In [ ]: clf, train_acc, test_acc = make_gridCV_classifier_print_errors()
```

```
SVM Train Accuracy: 0.96
SVM Test Accuracy: 1.00
```

1.4 Summary

Question 9: Summarize in the table given below all the training and testing accuracies you've had in the previous section. Give your thoughts on the performance of the various classifiers, and on using GridSearchCV to search for the right hyperparameters.

Method Training Accuracy Testing Accuracy	:-----: :-----: :-----:	Linear Regression
6.5675 6.6789	LR (2 var) 5.4770 5.6286	LR (3 var) 4.9782 5.6952
Naive Bayes 0.95 0.97	LinearSVC 0.91 0.87	
SVC (GridSearch) 0.96 1.00		

Just because training performance is good, does not mean testing performance is good. For LR (3 var), there is possible overfitting, when the testing error is higher than that of LR (2 var), even though its training accuracy is lower.

Using GridSearchCV makes it much easier to tune for the best hyperparameters. It is a methodical way to build and evaluate a model for each combination of hyperparameters.

Overall, the performance here may not translate to unseen data, since the hyperparameters were tuned on all the data here, and overfitting could occur doing so. This could be mitigated using a train/validation/test set split, where the model does not see the test set until the very end.

FOR LAB TA ONLY

Q0: ___/ 4

Q1: ___/ 2

Q2: ___/ 2

Q3: ___/ 2

Q4: ___/ 2

Q4 Code: ___/ 4

Q5: ___/ 2

Q6: ___/ 2

Q6 Code: ___/ 4

Q6 Code 2: ___/ 4

Q7: ___/ 2

Q8: ___/ 2

Q8 Code: ___/ 4

Q9: ___/ 4

Total: ___/ 40